Contact Centre Health Check

Application Support Guide

Installation · Configuration · Operations · Troubleshooting

Version	1.0
Product	Altron Health Check
Platform	Windows / Python 3.10+
Audience	IT Support & Operations

1. Architecture Overview

The Health Check application polls two contact centre platforms — AudioCodes (VoIP devices) and Genesys Cloud (edge infrastructure) — concurrently, merges the results into a single PDF report, and emails it to configured recipients. It runs as a native Windows Service on a schedule defined in the YAML configuration file.

Component Overview

Component	File	Purpose
Main Orchestrator	`main.py`	Entry point. Runs collectors, builds PDF, sends email.
AudioCodes Module	`healthcheck/audiocodes_alarm_reporter.py`	Polls AudioCodes devices via REST API for active and historical alarms.
Genesys Module	`healthcheck/genesys_health_reporter.py`	Polls Genesys Cloud via SDK for edge and trunk health.
PDF Merger	`healthcheck/pdf_merger.py`	Builds the unified PDF report from collected data.
Email Sender	`healthcheck/email_sender.py`	Sends the HTML summary email with PDF attachment.
Windows Service	`service.py`	Wraps main.py as a native Windows Service with scheduled triggers.
Service Installer	`install_service.py`	Installs, configures, and starts the Windows Service.
Configuration	`healthcheck_config.yaml`	Single YAML file controlling all settings.

Data Flow

On each scheduled trigger the service calls `main.run()`, which executes the following steps in order:

1	Load configuration from healthcheck_config.yaml
2	Run AudioCodes collector and Genesys collector concurrently (ThreadPoolExecutor)
3	Each collector polls its platform API, enriches data, and returns a data dict
4	pdf_merger.build() assembles the unified PDF from both data dicts
5	email_sender.send() emails the HTML summary with the PDF attached
6	Timestamped log file is written to the configured logDir

2. Prerequisites

Python

Python 3.10 or later is required. Python 3.9 will NOT work — the application uses union type hint syntax (X | Y) that was introduced in 3.10.

Download from **python.org/downloads**. During installation:

- Check **Add Python to PATH** on the first installer screen.
- On the final installer screen, click **Disable path length limit** — required for the Genesys SDK which has very long file paths.

Verify installation:

```
python --version pip --version
```

Python Packages

Package	Purpose
`requests`	AudioCodes REST API calls
`reportlab`	PDF generation
`pyyaml`	YAML config file parsing
`PureCloudPlatformClientV2`	Genesys Cloud SDK
`pywin32`	Windows Service framework
`schedule`	In-process job scheduling

Install all packages:

```
pip install requests reportlab pyyaml PureCloudPlatformClientV2 pywin32 schedule
```

Note: If pip reports a long path error during PureCloudPlatformClientV2 installation, enable Windows Long Path support first: run reg add "HKLM\SYSTEM\CurrentControlSet\Control\FileSystem" /v LongPathsEnabled /t REG_DWORD /d 1 /f in an Administrator terminal, then retry.

3. Installation

Folder Structure

Place all application files in a single directory. The recommended path is `C:\HealthCheck\` or similar — avoid paths inside OneDrive or other sync folders as file locking during sync can cause write errors.

```
C:\HealthCheck\ main.py service.py install_service.py healthcheck_config.yaml healthcheck\
__init__.py audiocodes_alarm_reporter.py genesys_health_reporter.py pdf_merger.py email_sender.py
```

First-Time Setup

Step	Action
1	Copy all application files to the install directory.
2	Edit healthcheck_config.yaml — fill in credentials, SMTP settings, device list, and output paths. See Section 4.
3	Install Python packages: pip install requests reportlab pyyaml PureCloudPlatformClientV2 pywin32 schedule
4	Test a manual run: cd C:\HealthCheck && python main.py
5	Verify a PDF was created in reportDir and a log in logDir.
6	Install the Windows Service (Administrator terminal): python install_service.py

4. Configuration Reference

All settings are stored in `healthcheck_config.yaml`. The file is divided into five namespaced sections. Always use forward slashes or double backslashes in file paths — single backslashes cause YAML parse errors.

Important: YAML is whitespace-sensitive. Always use spaces (not tabs) for indentation. String values containing special characters (#, :, @) must be quoted.

reporting:

Key	Type	Description	Example
`reportDir`	str	Directory where PDF reports are saved.	`"C:/HealthCheck/reports"`
`logDir`	str	Directory where timestamped log files are saved.	`"C:/HealthCheck/logs"`
`filenamePrefix`	str	Prefix for PDF filenames.	`"healthcheck_report"`
`logLevel`	str	Log verbosity: DEBUG, INFO, WARNING, ERROR.	`"INFO"`

audiocodes:

Key	Type	Description	Example
`authHeader`	str	Base64-encoded Basic auth header for AudioCodes REST API.	`"Basic "`
`historyDays`	int	Number of days of historical alarms to retrieve.	`7`
`maxWorkers`	int	Max parallel threads for device polling.	`10`
`devices`	list	List of AudioCodes device IP addresses.	`- "10.x.x.x"`
`deviceCountry`	dict	Map of IP address to country label for the PDF.	`"10.x.x.x": "Country"`

genesys:

Key	Type	Description	Example
`clientId`	str	Genesys Cloud OAuth2 client ID.	`"your-client-id"`
`clientSecret`	str	Genesys Cloud OAuth2 client secret.	`"your-secret"`

schedule:

Key	Type	Description	Example
`times`	list	List of daily run times in HH:MM format. Changes take effect on the next run without restarting the service.	`- "06:00" - "18:00"`

smtp:

Key	Type	Description	Example
`host`	str	SMTP server hostname or IP address.	`"10.x.x.x"`
`port`	int	SMTP port. 25 = relay, 587 = TLS, 465 = SSL.	`25`
`useTls`	bool	Set true to use STARTTLS (port 587).	`false`
`username`	str	SMTP username. Leave blank for unauthenticated relay.	`""`
`password`	str	SMTP password. Leave blank for unauthenticated relay.	`""`
`from`	str	Sender email address.	`"sender@company.com"`
`to`	str	Recipient address(es). Comma-separated for multiple.	`"ops@company.com"`
`subject`	str	Email subject line prefix.	`"Contact Centre Health Check"`

5. Windows Service

The application runs as a native Windows Service named **AltronHealthCheck**. It starts automatically on boot, runs as the SYSTEM account, and restarts automatically on failure. The service reads the schedule from the YAML config on every run.

Installing the Service

Open an **Administrator** Command Prompt, navigate to the install directory, and run:

```
cd C:\HealthCheck python install_service.py
```

The installer will check prerequisites, register the service, configure automatic failure recovery, and start the service immediately.

Service Management Commands

Action	Command
Check status	`sc query AltronHealthCheck`
Start service	`sc start AltronHealthCheck`
Stop service	`sc stop AltronHealthCheck`
Restart service	`sc stop AltronHealthCheck && sc start AltronHealthCheck`
Uninstall	`python install_service.py --remove`
Reinstall	`python install_service.py`
Verify in GUI	Win+R → services.msc → AltronHealthCheck

Note: All sc commands and install_service.py must be run from an Administrator terminal.

Failure Recovery

The installer configures automatic restart on failure: after 5 seconds on the first failure, 10 seconds on the second, and 30 seconds on any subsequent failure. The failure count resets after 24 hours of successful operation.

6. Running Manually

The application can be run directly from the command line without the Windows Service — useful for testing, debugging, or on-demand reports.

Basic Run

```
cd C:\HealthCheck python main.py
```

With a Custom Config File

```
python main.py --config C:\path\to\custom_config.yaml
```

Expected Output

A successful run produces:

- A PDF report in the configured `reportDir`
- A timestamped log file in the configured `logDir`
- An email sent to the configured recipients
- Console output showing progress (INFO level by default)

Note: Running manually while the service is also running is safe — each run is independent and writes to its own timestamped log and PDF file.

7. Scheduling

Run times are configured in the `schedule.times` section of `healthcheck_config.yaml`. The Windows Service re-reads the config on every scheduled run, so time changes take effect automatically without restarting the service.

Changing Run Times

1. Edit healthcheck_config.yaml:

```
schedule: times: - "06:00" # Morning run - "18:00" # Evening run
```

2. Save the file. The new times will be picked up on the next scheduled trigger — no service restart required.

Adding or Removing Run Times

Add or remove entries under `schedule.times`. Any number of times is supported. Times must be in HH:MM 24-hour format.

Behaviour on Service Start

When the service starts, it checks whether any configured time has already passed today. If so, it fires a health check immediately rather than waiting until the next day. This prevents missed runs after service restarts or system reboots.

8. Log Files

The application produces two distinct types of log file, both written to the directory configured in `reporting.logDir`.

File	Description	Retention
`healthcheck_service.log`	Service lifecycle events only: start, stop, scheduled triggers, service-level errors. Does not contain app run detail.	Rotating: 5MB max, 5 backups
`healthcheck_YYYYMMDD_HHMMSS.log`	Full detail for one health check run: device polling, alarm counts, PDF generation, email send result. One file per run.	Manual cleanup required

Log Levels

Level	Use case
`DEBUG`	Full detail — every alarm fetch, every API call. Logs grow large quickly. Use for troubleshooting only.
`INFO`	Default. Run summaries, device counts, alarm totals, unreachable devices.
`WARNING`	Failures and issues only. Minimal output, good for production monitoring.
`ERROR`	Critical failures only.

Change the level by editing `reporting.logLevel` in the YAML. Restart the service for the change to take effect.

9. Email Setup

The application sends an HTML email after every run containing a summary of the health check results and the full PDF report as an attachment.

SMTP Configuration

Scenario	Settings
Internal relay (no auth)	`host: relay-ip port: 25 useTls: false username: "" password: ""`
Office 365 / Exchange Online	`host: smtp.office365.com port: 587 useTls: true username: user@domain password: apppassword`
Gmail	`host: smtp.gmail.com port: 587 useTls: true username: user@gmail.com password: app-password`
Implicit SSL (port 465)	`host: mail.server.com port: 465 useTls: false (SSL is automatic on port 465)`

Multiple Recipients

Set `smtp.to` to a comma-separated list:

```
to: "ops@company.com, manager@company.com, support@company.com"
```

Email Content

Each email contains:

- Subject line with status tag: **OK**, **PARTIAL** (if one module failed), or **FAILED**
- AudioCodes section: device counts, active/historical alarm counts, critical alarm devices, unreachable devices
- Genesys Cloud section: edge/trunk counts, offline edges, out-of-service trunks
- PDF report attached

10. Adding a New AudioCodes Device

New devices are added by editing `healthcheck_config.yaml`. No code changes are required.

Steps

1. Add the device IP to the devices list:

```
audiocodes: devices: - "10.17.7.102" - "10.17.7.103" - "10.17.x.x" # new device
```

2. Add a country label to deviceCountry:

```
deviceCountry: "10.17.7.102": "Angola" "10.17.x.x": "Country Name" # new device
```

3. Save the file. The change takes effect on the next run — no restart needed.

Requirements

The device must be reachable from the machine running the service on port 443 (HTTPS). The same auth credentials (`audiocodes.authHeader`) are used for all devices. If a device uses different credentials, it cannot be added to the same instance without a code change.

Removing a Device

Remove the IP from both `devices` and `deviceCountry` in the YAML. The change takes effect on the next run.

11. Troubleshooting

Common Issues

Service won't start

Cause	Run sc query AltronHealthCheck — check the error code. Common cause: pywin32 not installed or Python not on PATH for SYSTEM account.
Fix	Reinstall with python install_service.py from an Administrator terminal. Verify with python -c "import win32serviceutil; print('OK')".

No PDF generated

Cause	Both collectors failed, or PDF generation threw an exception.
Fix	Check the timestamped run log in logDir. Look for ERROR entries. Set logLevel: DEBUG for full detail.

Email not sending — getaddrinfo failed

Cause	smtp.host is set to the placeholder value 'ip-address'.
Fix	Update smtp.host in healthcheck_config.yaml to the actual SMTP server IP or hostname.

Email not sending — authentication failed

Cause	SMTP credentials are incorrect.
Fix	Verify smtp.username and smtp.password. For Office 365, use an app password rather than the account password.

YAML parse error on startup

Cause	Invalid characters in config values — commonly a backslash in a file path or an unquoted @ or # character.
Fix	Use forward slashes in paths (C:/HealthCheck/logs). Quote all string values containing special characters. Validate with: python -c "import yaml; yaml.safe_load(open('healthcheck_config.yaml'))"

Device showing as unreachable

Cause	Network connectivity issue, firewall blocking port 443, or SSL certificate mismatch.
Fix	Test connectivity: curl -k https://device-ip/api/v1/status. Check firewall rules. For SSL errors, the device may need a firmware update.

Historical alarms showing Unknown date

Cause	The AudioCodes history endpoint returned stub records without timestamp fields. Detail fetch timed out.
Fix	This is normal for devices with slow API responses. The record is included with 'Unknown date' label. Increase timeout in fetch_alarm_detail if needed.

Long path error during pip install

Cause	Windows 260-character path limit exceeded during PureCloudPlatformClientV2 installation.
Fix	Enable long paths: run reg add "HKLM\SYSTEM\CurrentControlSet\Control\FileSystem" /v LongPathsEnabled /t REG_DWORD /d 1 /f in an Administrator terminal. Retry pip install.

Service log growing too large

Cause	logLevel set to DEBUG — generates one log line per alarm per device per run.
Fix	Set logLevel: INFO in the config and restart the service.

TypeError: unsupported operand type for |

Cause	Python version is below 3.10.
Fix	Upgrade Python to 3.10 or later. See Section 2.

12. Quick Reference — Diagnostic Commands

Task	Command
Check service status	`sc query AltronHealthCheck`
View service log	`type C:\HealthCheck\logs\healthcheck_service.log`
Validate YAML config	`python -c "import yaml; yaml.safe_load(open('healthcheck_config.yaml')); print('OK')"`
Check Python version	`python --version`
Check installed packages	`pip list`
Run manually	`cd C:\HealthCheck && python main.py`
Run with debug logging	`Set logLevel: DEBUG in YAML then run python main.py`
Test device connectivity	`curl -k https://10.x.x.x/api/v1/status`
Enable long paths	`reg add "HKLM\SYSTEM\CurrentControlSet\Control\FileSystem" /v LongPathsEnabled /t REG_DWORD /d 1 /f`
Reinstall service	`python install_service.py --remove && python install_service.py`